Order Book Requirements

What it does

- Simulates a simplified financial exchange's order book.
- Tracks outstanding buy orders (bids) and sell orders (asks).
- Matches compatible buy and sell orders and records executed trades.
- Updates or removes orders as they are filled or cancelled.
- Uses **price-time priority** to determine which orders get executed first.

1. Order representation

Each order needs to store:

- **Order ID:** uniquely identifies the order.
- **Side:** buy or sell.
- **Price:** the limit price the trader is willing to trade at.
- **Quantity:** the number of shares/contracts remaining to be filled.
- **Time priority:** determines when the order was accepted relative to other orders.

Implementation considerations: - Represent an order using an `Order` struct or class. - Use `enum class Side { Buy, Sell }` rather than an integer to represent the side. - Use integer price ticks (e.g. £10.25 = 1025 if one tick represents £0.01) rather than `double` to avoid floating-point precision issues. - Use an integer sequence number to establish deterministic time priority. - Track remaining quantity because an order can be partially filled.

2. Storing orders

- Maintain separate collections for buy and sell orders.
- Buy orders are prioritised by **highest price first**.
- Sell orders are prioritised by **lowest price first**.
- If multiple orders have the same price, the earliest accepted order has priority (FIFO — first in, first out).
- Quantity does not determine priority in a price-time-priority system.

3. Matching and executing orders

A trade can occur when the best buy price is greater than or equal to the best sell price.

When compatible orders meet: - Execute the minimum of the two orders' remaining quantities. - Record the trade, including buyer ID, seller ID, execution price and quantity. - Reduce the remaining quantity of both orders by the executed quantity. - Remove any fully filled orders from the book. - Keep partially filled orders in the book with their remaining quantity. - Continue matching if the incoming order still has quantity remaining and compatible orders are available.

Execution price rule: for this project, use the resting order's price as the execution price.

4. Cancelling orders

- Allow an outstanding order to be cancelled by its order ID.
- Remove the remaining quantity from the book when an order is cancelled.

- Do not allow already completed or unknown orders to be cancelled successfully.
- A partially filled order can be cancelled, but only its remaining quantity is removed.

5. Initial class design

`Order`

Responsible for representing an individual order.

Potential data members: - Order ID - Side - Price - Remaining quantity - Arrival sequence number

`OrderBook`

Responsible for managing outstanding orders and applying the matching rules.

Potential responsibilities: - Add new orders. - Store and prioritise buy and sell orders. - Match compatible orders. - Update remaining quantities. - Remove fully filled orders. - Cancel outstanding orders. - Provide the best bid and best ask. - Calculate the bid-ask spread. - Store and expose completed trade records.

Start with these two types. Additional classes, such as `Trade` or `PriceLevel`, can be introduced when they provide a clear benefit.

6. Core matching rules

1. Highest buy price has priority.
2. Lowest sell price has priority.
3. Earlier arrival wins when prices are equal.
4. Matching continues while the best bid is greater than or equal to the best ask.
5. Each execution uses the smaller of the two remaining quantities.
6. Fully filled orders are removed; partially filled orders remain.
7. Each execution is recorded as a trade.

7. Development plan

1. Implement the `Order` struct and test creating orders.
2. Create the `OrderBook` class and store buy and sell orders.
3. Implement price-time priority and identify the best bid and ask.
4. Implement matching for a single buy and sell order.
5. Add partial fills and matching across multiple orders.
6. Implement order cancellation.
7. Record trades and expose basic order book information.
8. Write unit tests for matching, priority, cancellation and edge cases.
9.